

Week Module - 04

Time & Space complexity (Asymptotic Analysis)

• print Numbers 1 to N

Time complexity $\rightarrow O(N)$

Space complexity $\rightarrow O(1)$

Explanation $\rightarrow$ loop executes N times & prints each number once.

• Sum of N natural number (loop)

Time complexity $\rightarrow O(N)$

Space complexity $\rightarrow O(1)$

Explanation $\rightarrow$ loop adds N number one by one using constant memory.

• Sum using Formula.

Time complexity $\rightarrow O(1)$

Space complexity $\rightarrow O(1)$

Explanation $\rightarrow$ we generally here use the mathematical formula to solve without using looping.

• power A^B (loop)

Time complexity $\rightarrow O(B)$

Space complexity $\rightarrow O(1)$

Explanation $\rightarrow$ Performs B multiplication.

• power using Binary Exponentiation

Time complexity $\rightarrow O(\log B)$

Space complexity $\rightarrow O(1)$

Explanation $\rightarrow$ Exponent is halved in each iteration, reducing operation significantly.

(Section - A)

week - 1

• Square pattern $\rightarrow$ T.C $\rightarrow O(n^2)$
S.C $\rightarrow O(1)$

Explanation $\rightarrow$ two nested loops print $n \times n$ characters.

• Hollow Square -

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

Explanation $\rightarrow$ Every row & column is visited once; only border elements are printed.

• Triangle pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

The total number of printed element is $n(n+1)/2$ which is $O(n^2)$

• Inverted Triangle

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

$\rightarrow$ uses nested loops with decreasing iterations, but total work is still $O(n^2)$!

• Pyramid pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

$\rightarrow$ Spaces & stars are printed using nested loops for each row.

• Diamond pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

$\rightarrow$ upper & lower pyramid together require quadratic work.

• Half Diamond.

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

prints increasing & decreasing rows, total operations are quadratic.

• Inverted Half Diamond.

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

similar to Half Diamond with reverse order.

• Number square pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

prints $n \times n$ numbers.

• Incrementing Number patterns:-

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

$\rightarrow$ Due to the use of nested loop to print no. in each row.

• Right Triangle No. pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

Total printed number are $n(n+1)/2$

• Inverted Diamond Number pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

Multiple nested loops prints spaces and number.

• Sandwich pattern

T.C $\rightarrow O(n^2)$

S.C $\rightarrow O(1)$

Requires nested loops for each row.

• Incrementing Diamond pattern

S.C $\rightarrow O(1)$

T.C $\rightarrow O(n^2)$

Combination of upper & lower parts gives quadratic complexity.

Week - 2 & 3

• Linear Search $\rightarrow$

Best time $\rightarrow O(1)$

Avg. time $\rightarrow O(n)$

Worst time $\rightarrow O(n)$

Space complexity $\rightarrow O(1)$

Explanation:-

$\rightarrow$ If the element is found at the first position, only one comparison is needed. In the avg. & worst case it may scan the entire array.

Binary search

Best time $\rightarrow O(1)$
Avg. time $\rightarrow O(\log n)$
Worst time $\rightarrow O(\log n)$
S.C $\rightarrow O(1)$ (iterative)
 $O(\log n)$ (recursive)

Explanation

The search space is reduced by half after every comparison, making it much faster than linear search on sorted data.

Bubble sort

Best time $\rightarrow O(n)$
Avg. time $\rightarrow O(n^2)$
Worst time $\rightarrow O(n^2)$
S.C $\rightarrow O(1)$

$\rightarrow$ Best case occurs when the array is already sorted otherwise repeated passes compare adjacent elements.

Insertion sort

Best time $\rightarrow O(n)$
Avg. time $\rightarrow O(n^2)$
Worst time $\rightarrow O(n^2)$
S.C $\rightarrow O(1)$

$\rightarrow$ Efficient for small or nearly sorted array because each element is inserted into its correct position.

Selection sort

Best time $\rightarrow O(n^2)$
Avg. time $\rightarrow O(n^2)$
Worst time $\rightarrow O(n^2)$
S.C $\rightarrow O(1)$

$\rightarrow$ It always searches the remaining unsorted part to find the minimum element, regardless of input order.

Merge sort

Best time $\rightarrow O(n \log n)$
Avg. time $\rightarrow "$
Worst time $\rightarrow "$
S.C $\rightarrow O(n)$

$\rightarrow$ The array is divided into halves recursively & then merged. T.C remains the same in all cases, but extra $n/\ln$ is required for merging.

quick sort

Best time $\rightarrow O(n \log n)$

Avg. time $\rightarrow "$

worst time $\rightarrow "$

S.C $\rightarrow O(\log n)$

$\rightarrow$ quick sort partitions the array around a pivot. If the pivot is poor (e.g. $\rightarrow$ always the smallest / largest) the worst case becomes $O(n^2)$.

Heap sort

Best time $\rightarrow O(n \log n)$

Avg. time $\rightarrow "$

worst time $\rightarrow "$

S.C $\rightarrow O(1)$

$\rightarrow$ A heap is built first, then the max/min element is repeatedly removed. It provides guaranteed $O(n \log n)$ performance with constant extra space.

Problem Statement

- ① In this question generally we have to do on thing that take a positive integer N & print all the number from 1 to N in increasing order.

```
→ #include <iostream>
    using namespace std;
    int main() {
        int N;
        cin >> N;
        for (int i = 1; i <= N; i++) {
            cout << i << " ";
        }
        return 0;
    }
```

Time complexity :- $O(N)$

- Because the loop runs exactly N times.
- Generally in the iteration of the loop total execution time increases linearly.
- In this we also use only one integer variable (i) so, memory is proportional to N .

Space complexity :- $O(1)$

- means no extra space is required.

Optimized Approach

- There is no faster algorithm because every no. from 1 to N must be printed.

Problem - ② :- In this we have to sum the first N natural numbers using a loop.

→ #include <iostream>
using namespace std;

int main() {

long long N, sum = 0;

cin >> N;

for (long long i = 1; i <= N; i++) {

sum += i;

}

cout << sum;

return 0;

}

→ Here we can solve by formula also
$$\left(\frac{N(N+1)}{2} \right)$$

Time complexity :- $O(N)$

- Because the loop will be executed N times.
- Each iteration performs one addition operation.

Space complexity :- $O(1)$

- Because only two variables (i & sum) are used. So, generally fixed memory size is required.

Optimized Approach

- Instead of using a loop, we can solve this through the formula —

$$sum = \frac{N(N+1)}{2}$$

And when we use this the time & space complexity will be — $O(1)$

Problem - 03. Calculate A^B using a loop.

```
#include <iostream>
using namespace std;
int main() {
    long long A, B;
    cin >> A >> B;
    long long result = 1;
    for (int i = 1; i <= B; i++) {
        result *= A;
    }
    cout << result;
    return 0;
}
```

Time complexity :- $O(B)$

- The loop will run exactly B times.
- Each iteration multiplies the current result by A .

Space complexity :- $O(1)$

- Generally here few variables are used, so the extra memory remains constant.

Optimized Approach

- Generally the efficient way is that we can solve through Binary Exponentiation (Fast power).

ine_c++_compiler#



main.cpp

```
1 // HERE WE ARE JUST PERFORMING THE CODE TO PRINT THE NUMBER FROM 1 TO N
2 #include<iostream>
3 using namespace std;
4 int main(){
5     int N;
6     cin>>N;
7     for(int i=1; i<=N;i++){
8         cout<<i<<" ";
9     }
10    return 0;
11 }
```

input

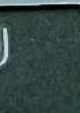
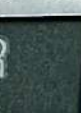
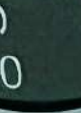
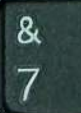
```
5
1 2 3 4 5
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Contact
cy



Search



ie_c++_compiler#

Run Debug Stop Share Save Beautify

main.cpp

```
1 // HERE IN THIS WE HAVE SUM THE FIRST N NATURAL NUMBER USING A LOOP
2 #include<iostream>
3 using namespace std;
4 int main(){
5     long long N,sum=0;
6     cin>>N;
7     for(long long i=1; i<=N; i++){ // here we can also use the form to solve without loop
8         sum+=i;
9     }
10    cout<<sum;
11    return 0;
12 }
```

input

5
151

...Program finished with exit code 0
Press ENTER to exit console.

Contact
Privacy
e



Search



F3

F4

F5

F6

F7

F8

F9

F10

F11

F12

Insert

PrtSc

#

\$

%

^

&

*

(

)

-

=

B

3

4

5

6

7

8

9

0

-

=

B

E

R

T

Y

U

I

O

P

[

]

B

D

F

G

H

J

K

L

:

"

'

B

X

C

V

B

N

M

<

>

?

/

B

X

C

V

B

N

M

<

>

?

/

B

X

C

V

B

N

M

<

>

?

/

B

X

C

V

B

N

M

<

>

?

/

B

X

C

V

B

N

M

<

>

?

/

B

com/online_c++_compiler#

Run Debug Stop Share Save Beautify

main.cpp

```
1 // HERE WHAT WE CALCULATE A AND B USING LOOP
2 #include<iostream>
3 using namespace std;
4 int main(){
5     long long A,B;
6     cin>>A>>B;
7     long long result = 1;
8     for(int i=1;i<=B;i++){
9         result*=A;
10    }
11    cout<<result;
12    return 0;
13 }
```

input
3 4
81
...Program finished with exit code 0
Press ENTER to exit console.

Terms of Use • Contact
Edits • Privacy
DB Online



Search

